Experimental Lab 1: Measuring Time Complexity

Lab Objective:
This lab will guide you through evaluating time complexity using Big O notation experimentally. You will implement several functions with different time complexities and measure their execution time for different input sizes.

Lab Setup:

- **Language**: C++
- **Tools**: IDE or any C++ compiler (e.g., **Visual Studio**)
- **Library**: <chrono> for time measurement

Tasks Overview:

Task 1: Implement a Function with Constant Time Complexity O(1)

1. **Goal**: Implement a function that runs in constant time, regardless of the input size n.
2. **Steps**:
    - Define a function constantFunction(int n) that performs a single arithmetic operation on n, like multiplying it by 2.
    - Ensure the function does not depend on the input size n.
3. **Expected Behavior**: The time taken by this function should remain the same for any input size.

Task 2: Implement a Function with Linear Time Complexity O(n)

1. **Goal**: Implement a function that runs in linear time, where the time complexity increases in direct proportion to n.
2. **Steps**:
    - Define a function linearFunction(int n) that uses a loop to iterate n times.
    - Inside the loop, perform a simple operation, like multiplying the index by 2.
3. **Expected Behavior**: The time taken by this function should increase linearly with n.

Task 3: Implement a Function with Quadratic Time Complexity O(n2)

1. **Goal**: Implement a function that runs in quadratic time, meaning its execution time grows with the square of n.
2. **Steps**:
    - Define a function quadraticFunction(int n) that contains two nested loops, each iterating n times.

- Inside the inner loop, perform an operation, such as multiplying the loop variables.

3. **Expected Behavior**: The time taken by this function should grow exponentially as the input size increases.

Task 4: Implement a Function with Logarithmic Time Complexity O(log⁡n)

1. **Goal**: Implement a function that runs in logarithmic time, meaning the number of operations grows in proportion to the logarithm of n.
2. **Steps**:
   - Define a function logarithmicFunction(int n) that repeatedly divides n by 2 in a loop.
   - The loop should continue until the input size is reduced to 1.
3. **Expected Behavior**: The time taken by this function should grow logarithmically with n.

Task 5: Measuring Execution Time for Each Function

1. **Goal**: Measure and compare the execution time of the functions you implemented in Tasks 1-4.
2. **Steps**:
   - Use the <chrono> library to measure the time taken by each function for different input sizes.
   - Create a helper function evaluate that measures the execution time of a given function.

- Test the functions with various input sizes, such as n=10,100,1000,5000,10000n

Task 6: Analysis and Reporting

1. **Goal**: Analyze the results and compare the empirical measurements with the theoretical Big O and Big Omega notations.
2. **Steps**:
   - Create a table that records the execution times for each function at different input sizes.
   - Compare the results with the theoretical expectations of Big O and Big Omega.
   - Reflect on whether the results align with the theoretical complexities.

Input Size (n)	O(1)	O(n)	O(n2)	O(logn)
100				
1000				
10000				
100000				

Hint: To measure the running time, you can use the following code:

```
auto start = high_resolution_clock::now();
Function(n);
auto stop = high_resolution_clock::now();
auto duration = duration_cast<microseconds>(stop - start);
```